

Relazione Progetto: File System Statistics (FSStat)

Programmazione Concorrente e Distribuita

Alessandro Martini, Alessandro Torelli

Anno Accademico 2025/2026

Sommario

Il presente documento descrive la progettazione e l'implementazione della libreria **FSStatLib**, sviluppata in Java 25. L'obiettivo del progetto è analizzare un albero di directory del file system, calcolando il numero totale di file e la loro distribuzione in fasce di dimensione. L'elaborazione è stata parallelizzata utilizzando tre metodi e tecnologie differenti: i Virtual Threads, programmazione asincrona basata su event loop, e la programmazione reattiva con RX.

Indice

1	Introduzione e Requisiti	2
2	Analisi del Problema e Scelte Concorrenti	2
2.1	Approccio con i Virtual Threads	3
2.2	Approccio Asincrono basato su Event Loop	4
2.3	Approccio Reattivo con RX	5
3	Architettura e Design	5
3.1	Design dell'implementazione Virtual Threads	5
3.2	Design dell'implementazione Event Loop	7
3.3	Design dell'implementazione RX	9

1 Introduzione e Requisiti

Obiettivo

L'obiettivo è la realizzazione della libreria `FSStatLib`, che espone un metodo asincrono `getFSReport` per l'analisi statistica delle dimensioni dei file contenuti in un albero di directory del file system. Dato un percorso radice `D`, il metodo calcola e restituisce in modo asincrono un report `R` contenente:

- il **numero totale di file** presenti in `D`, incluse ricorsivamente tutte le sottodirectory;
- la **distribuzione delle dimensioni** dei file: dato un valore massimo di dimensione `MaxFS` e un numero di bande `NB`, il range `[0, MaxFS]` viene suddiviso in `NB` intervalli di ampiezza uniforme; per ciascun intervallo viene contato il numero di file la cui dimensione vi rientra. Viene inoltre conteggiata separatamente la quantità di file con dimensione superiore a `MaxFS`. Il report include quindi complessivamente `NB + 1` contatori.

I parametri `D`, `MaxFS` e `NB` sono forniti dal chiamante come argomenti del metodo.

Versioni implementate

Il progetto è stato declinato in **tre versioni indipendenti**, ciascuna aderente a una specifica disciplina di programmazione concorrente discussa durante il corso:

1. **Virtual Threads** (Java): paradigma *thread-per-task* con Virtual Thread leggeri gestiti dalla JVM.
2. **Programmazione asincrona basata su Event Loop**: composizione di operazioni non bloccanti tramite `CompletableFuture` e callback.
3. **Programmazione reattiva con RX**: modellazione del file system traversal come stream di eventi, elaborato tramite operatori funzionali (RxJava).

Un vincolo di progetto esplicito impone che le tre versioni siano sviluppate in modo **indipendente**, senza generalizzare o riutilizzare codice tra di esse qualora ciò compromettesse la purezza della disciplina di programmazione propria di ciascun approccio.

Requisito opzionale

Il progetto prevede un requisito opzionale `[*]`, obbligatorio per il conseguimento della lode, che estende la libreria con funzionalità interattive:

- possibilità di **interrompere** la generazione del report in qualsiasi momento;
- **aggiornamenti dinamici** delle statistiche durante la computazione, visualizzabili in tempo reale.

A supporto di queste funzionalità è prevista una **GUI minimale**, dotata di controlli per avviare e interrompere la scansione e di un'area di visualizzazione per il monitoraggio progressivo del report.

2 Analisi del Problema e Scelte Concorrenti

L'attraversamento ricorsivo di un file system e l'interrogazione delle proprietà dei file sono tipiche operazioni legate all'I/O. In questi scenari di I/O-bound, l'uso intensivo dei classici thread di piattaforma presenta gravi limiti di scalabilità. Per esplorare soluzioni diverse a questo problema di concorrenza, il progetto è stato declinato in tre paradigmi distinti.

2.1 Approccio con i Virtual Threads

I Virtual Threads, introdotti stabilmente in Java 21, sono thread *user-mode* gestiti dalla JVM anziché dal sistema operativo. Rispetto ai *platform thread*, il loro overhead di creazione e il consumo di memoria sono trascurabili, il che li rende adatti a un modello *Thread-per-task*: è possibile istanziarne decine di migliaia senza incorrere in `OutOfMemoryError` né in degrado delle prestazioni.

Questa caratteristica si adatta in modo naturale all'attraversamento ricorsivo di un albero di directory: la struttura del file system è intrinsecamente ramificata e imprevedibile, e ogni nodo (directory) richiede operazioni di I/O bloccanti a latenza variabile (apertura dello stream, lettura degli attributi dei file con `Files.size()`). Assegnare un Virtual Thread a ciascun nodo permette di sfruttare appieno la concorrenza senza dover dimensionare a priori un pool.

Struttura dell'implementazione. Il punto di ingresso pubblico è il metodo `getFSReport()`, che restituisce una `CompletableFuture<FSReport>` per offrire un'interfaccia non bloccante al chiamante (CLI o GUI). Internamente, la computazione reale viene avviata tramite `CompletableFuture.supplyAsync()` che esegue il corpo su un Virtual Thread. All'interno di questo contesto viene creato un `ExecutorService` dedicato tramite `Executors.newVirtualThreadPerTaskExecutor()`, racchiuso in un blocco `try-with-resources`:

```
1 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
2     Path rootPath = Paths.get(directoryPath);
3     return processDirectory(rootPath, maxFS, nb, executor).get();
4 }
```

La chiusura automatica dell'executor al termine del blocco `try` ha una semantica cruciale: `ExecutorService.close()` invoca `shutdown()` seguito da `awaitTermination()`, garantendo che nessun task figlio venga abbandonato prima che il risultato finale sia disponibile. Questo meccanismo funge da *barriera di sincronizzazione* implicita su tutto l'albero dei task.

Ricorsione concorrente e Map-Reduce. Il metodo `processDirectory()` incarna il pattern *divide et impera*: per ogni nodo dell'albero viene sottomesso all'executor un nuovo task (e quindi un nuovo Virtual Thread). All'interno del task, si itera sul contenuto della directory tramite `Files.newDirectoryStream()`:

- Per ogni **file regolare**: si invoca `Files.size()` (operazione I/O bloccante) e si aggiorna un oggetto `FSReport locale` al Virtual Thread corrente.
- Per ogni **sottodirectory**: si sottomette ricorsivamente un nuovo task all'executor, raccogliendo il `Future<FSReport>` restituito in una lista di *subTask*.

Terminata la scansione della directory corrente, il Virtual Thread chiama `task.get()` su ciascun *subTask* per attenderne il completamento e fondere i risultati parziali nel report locale tramite il metodo `FSReport.merge()`:

```
1 for (Future<FSReport> task : subTasks) {
2     localReport.merge(task.get());
3 }
```

Questa fase di riduzione segue un pattern *Map-Reduce*: la fase di *map* distribuisce il lavoro di scansione su Virtual Thread indipendenti, mentre la fase di *reduce* aggrega i report parziali bottom-up lungo l'albero, propagando i risultati verso il nodo radice.

Assenza di contesa e gestione dell'I/O bloccante. Una proprietà rilevante di questo design è la completa assenza di stato condiviso mutabile tra Virtual Thread concorrenti: ogni task opera esclusivamente sul proprio `FSReport` locale. La fusione dei risultati (`merge`) avviene sempre in modo sequenziale nel Virtual Thread padre, dopo che i figli sono terminati. Ne consegue che non è necessario alcun meccanismo di sincronizzazione esplicito (né `synchronized` né `ReentrantLock`) per proteggere il report.

Quando un Virtual Thread si blocca su un'operazione di I/O — ad esempio `Files.size()` o `task.get()` in attesa di un risultato non ancora disponibile — la JVM effettua automaticamente l'*unmount* del Virtual Thread dal *carrier thread* (il platform thread sottostante), che viene immediatamente liberato per eseguire un altro Virtual Thread pronto. Questo meccanismo permette al ridotto pool di carrier thread (tipicamente uguale al numero di core fisici) di servire migliaia di Virtual Thread concorrenti, eliminando il costo del context switch a livello di sistema operativo.

2.2 Approccio Asincrono basato su Event Loop

Il secondo paradigma adottato si basa sul modello *Event Loop*, incarnato in questa implementazione dal runtime **Node.js** (scritto in TypeScript). In questo modello, un **singolo thread JavaScript** esegue il codice applicativo, delegando tutte le operazioni di I/O al layer sottostante *libuv*, che le gestisce tramite un proprio thread pool interno al runtime. Al completamento di ciascuna operazione, la relativa callback viene accodata nell'event loop ed eseguita dal thread principale non appena questo è libero.

Adattamento al problema. L'attraversamento ricorsivo di un albero di directory è un workload *I/O-bound* con un elevato grado di parallelismo naturale: i contenuti di directory diverse sono indipendenti tra loro e possono essere letti concorrentemente. Questo si adatta perfettamente al modello event loop, dove la concorrenza non è ottenuta tramite thread multipli ma tramite l'*interleaving* di operazioni asincrone: il thread principale non attende mai il completamento di una lettura su disco, ma registra una `Promise` e prosegue a processare altri eventi già completati.

Concorrenza tramite `Promise.all`. Il cuore della concorrenza risiede nel metodo `traverseDirectory` di `FsStatScannerImpl`. Dopo aver letto il contenuto di una directory tramite l'API non bloccante `fs.readdir()` (da `fs/promises`), il metodo separa i risultati in file e sottodirectory, quindi lancia *simultaneamente* tutte le operazioni figlie tramite `Promise.all`:

```
1 await Promise.all([
2   ...files.map((file) => exploreFile(file.name)),
3   ...directories.map((dir) => exploreDir(dir.name)),
4 ]);
```

Questo produce un *fan-out* parallelo: tutte le chiamate a `fs.stat()` sui file e tutte le esplorazioni ricorsive delle sottodirectory vengono avviate in parallelo, senza attendere che una termini prima di avviare la successiva. La ricorsione si propaga in profondità nell'albero mantenendo questo stesso pattern ad ogni livello.

Assenza di sincronizzazione per costruzione. Una proprietà fondamentale di questo approccio è che l'aggiornamento dello stato condiviso — il metodo `updateReport(fileSize)` su `FsStatReportImpl` — avviene **senza alcun meccanismo di sincronizzazione** (niente lock, niente mutex, niente operazioni atomiche). Questo è sicuro per costruzione grazie alla *run-to-completion semantics* di JavaScript: l'event loop garantisce che ogni callback venga eseguita interamente prima che un'altra possa iniziare. Due invocazioni di `updateReport` non possono mai sovrapporsi temporalmente, rendendo strutturalmente impossibile qualsiasi race condition sullo stato del report.

Gestione degli errori e robustezza. L'implementazione distingue due categorie di errori di I/O. Gli errori di accesso (EACCES, EPERM) vengono silenziosamente ignorati tramite `.catch()`, permettendo alla scansione di proseguire sui nodi raggiungibili. Gli errori strutturali (ENOENT, ENOTDIR) vengono invece convertiti in eccezioni tipizzate e propagati al chiamante, causando il rigetto della `Promise` restituita da `getReport`. È inoltre presente una *blacklist* di path di sistema (`/proc`, `/sys`, `/dev` e altri) che vengono esclusi preventivamente dalla scansione per evitare loop infiniti o comportamenti non deterministici su filesystem virtuali.

2.3 Approccio Reattivo con RX

La terza tecnologia utilizza la Programmazione Reattiva (es. RxJava), un paradigma orientato ai flussi di dati (Data Streams) e alla propagazione dei cambiamenti. L'attraversamento del file system è stato modellato come uno stream asincrono di eventi (file trovati o directory aperte) emessi da un *Publisher / Observable*. L'approccio reattivo ha permesso di operare in modo puramente dichiarativo: attraverso operatori funzionali (`map`, `flatMap`, `filter`, `scan`) gli eventi vengono trasformati e aggregati al volo per costruire il report. Questo approccio è intrinsecamente non bloccante e facilita enormemente la gestione degli aggiornamenti in tempo reale per la GUI.

3 Architettura e Design

Il progetto è suddiviso seguendo il principio di Separazione delle Responsabilità (Separation of Concerns). Oltre al livello dell'interfaccia (CLI e GUI), la logica di dominio (il package `lib`) è stata strutturata per accomodare le tre diverse implementazioni tramite un'interfaccia comune, permettendo al chiamante di scegliere il motore di concorrenza preferito.

3.1 Design dell'implementazione Virtual Threads

L'implementazione con Virtual Threads è articolata in due classi principali nel package `it.unibo.fsstat.lib`: `FSReport`, che modella il risultato della computazione, e `FSStatLib`, che ne orchestra l'esecuzione concorrente. La CLI (`Main`) interagisce esclusivamente con l'interfaccia pubblica di `FSStatLib`, rimanendo ignara del paradigma di concorrenza sottostante.

Il modello dati: FSReport. `FSReport` è un *value object* che incapsula tre informazioni: il conteggio totale dei file (`totalFiles`), i parametri di classificazione (`maxFS` e `nb`), e un array `distribution` di dimensione `nb + 1`. Le prime `nb` celle corrispondono alle bande di dimensione nell'intervallo `[0, maxFS]`, mentre la cella di indice `nb` raccoglie i file che superano `maxFS`.

La classificazione di un singolo file avviene nel metodo `addFile(long size)`, che calcola l'indice di banda tramite divisione intera:

```
1 double step = (double) maxFS / nb;
2 int bandIndex = (int) (size / step);
3 if (bandIndex == nb) bandIndex = nb - 1; // Caso limite: size == maxFS
```

Il metodo `merge(FSReport other)` somma componente per componente i due array `distribution` e accumula i conteggi di `totalFiles`. Questa operazione è la chiave del pattern Map-Reduce: è associativa e non richiede alcuna sincronizzazione perché viene sempre invocata in modo sequenziale dal Virtual Thread padre dopo che i figli sono terminati.

La libreria: FSStatLib. L'interfaccia pubblica espone un unico metodo:

```
1 public CompletableFuture<FSReport> getFSReport(
2     String directoryPath, long maxFS, int nb)
```

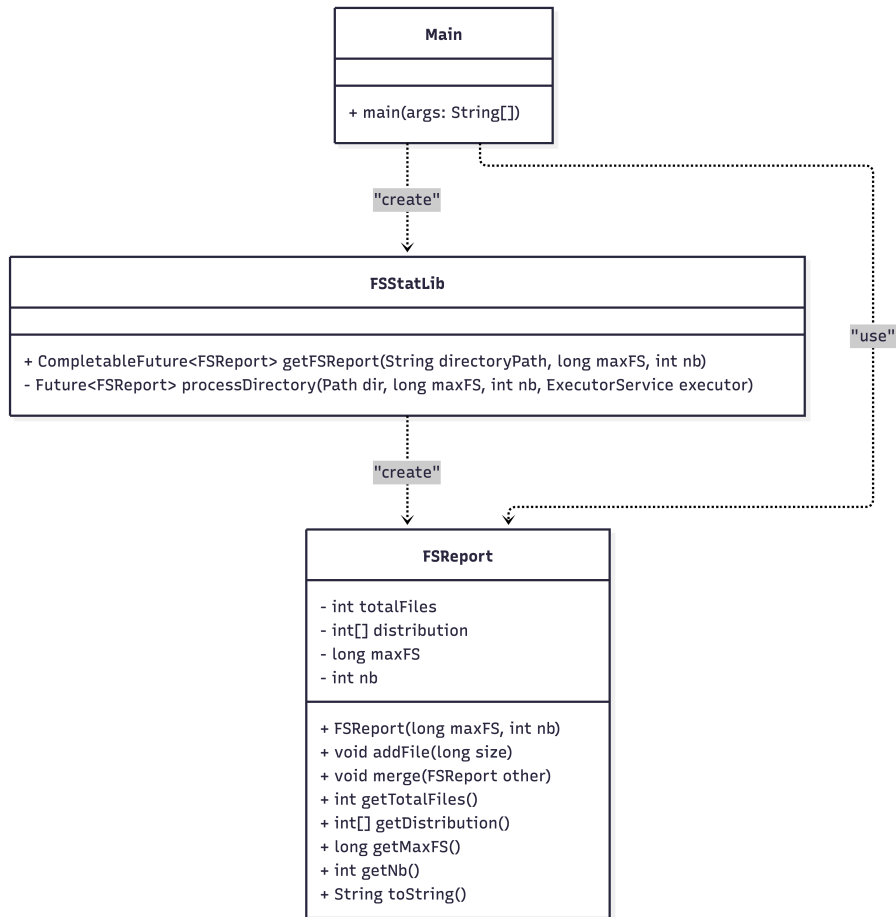


Figura 1: Diagramma UML delle classi dell'implementazione con Virtual Threads.

La restituzione di una `CompletableFuture` disaccoppia il chiamante dalla concorrenza interna: la CLI può concatenare callback (`thenAccept`) oppure attendere sincronamente con `join()`, senza dover conoscere né i Virtual Thread né l'executor sottostante.

Internamente, `getFSReport` avvia la computazione tramite `CompletableFuture.supplyAsync()`, che esegue il corpo su un thread del `ForkJoinPool` comune. Da questo contesto viene creato l'executor dedicato ai Virtual Thread:

```

1 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
2     Path rootPath = Paths.get(directoryPath);
3     return processDirectory(rootPath, maxFS, nb, executor).get();
4 }

```

Il blocco `try-with-resources` ha una funzione di lifecycle management precisa: alla sua uscita, `ExecutorService.close()` invoca `shutdown()` e `awaitTermination()`, assicurando che tutti i Virtual Thread lanciati ricorsivamente abbiano completato la propria esecuzione prima che il risultato venga restituito alla `CompletableFuture`.

Il flusso ricorsivo: `processDirectory`. `processDirectory` è il cuore dell'algoritmo. Ogni sua invocazione sottomette all'executor un nuovo task — e quindi un nuovo Virtual Thread — incaricato di elaborare una singola directory. Il flusso interno al task segue tre fasi:

1. **Scansione locale:** si itera sul contenuto della directory tramite `Files.newDirectoryStream()`. Per ogni file regolare, `Files.size()` legge la dimensione dal disco (operazione I/O bloccante) e la classifica nel `FSReport` locale al thread. Per ogni sottodirectory, si invoca

ricorsivamente `processDirectory`, raccogliendo il `Future<FSReport>` restituito in una lista (`subTasks`).

2. **Riduzione (join):** terminata la scansione, il Virtual Thread chiama `task.get()` su ciascun elemento di `subTasks`. Se il task figlio non è ancora completato, il Virtual Thread si sospende — effettuando l'*unmount* dal carrier thread — e si risveglia non appena il risultato è disponibile. I report parziali vengono fusi nel report locale tramite `merge()`.
3. **Restituzione:** il report locale, ora completo di tutti i contributi dei sottoalberi, viene restituito come valore del `Callable` sottomesso all'executor.

La struttura risultante è un albero di Virtual Thread isomorfo all'albero delle directory: ogni nodo dell'albero FS corrisponde a un task concorrente, e la riduzione avviene bottom-up per propagazione dei `Future`.

Gestione degli errori. Gli errori di accesso al file system (directory protette, symlink non risolvibili) vengono catturati localmente da un blocco `catch (IOException e)` all'interno di ciascun task: la directory inaccessibile viene saltata con un messaggio su `stderr`, mentre la computazione prosegue sui nodi raggiungibili. Eventuali eccezioni non gestite che sfuggono al task vengono incapsulate in una `CompletionException` e propagate al chiamante tramite la `CompletableFuture`, che entra nello stato *exceptionally completed* e può essere intercettata con `.exceptionally()`.

Integrazione con la CLI. In `Main`, il chiamante configura i tre parametri della scansione (`directoryPath`, `maxFS`, `nb`) e concatena alla `CompletableFuture` una callback `thenAccept` per la stampa del report. La chiamata `.join()` finale blocca il thread principale fino al completamento, evitando che il programma termini prima che la computazione asincrona sia conclusa. La formattazione del report calcola le etichette delle bande moltiplicando l'indice per `bandSize = maxFS / nb`, producendo intervalli del tipo `[i·bandSize, (i+1)·bandSize]`.

3.2 Design dell'implementazione Event Loop

L'implementazione con Event Loop è sviluppata in **TypeScript** sul runtime **Node.js**, ed è organizzata in moduli con responsabilità ben separate. A differenza della versione Java con Virtual Threads, l'intera concorrenza è ottenuta senza istanziare alcun thread esplicitamente: il parallelismo emerge dalla composizione di `Promise` gestite dall'event loop di Node.js e dal thread pool di *libuv*.

Struttura dei moduli. Il codebase è suddiviso in moduli distinti, ciascuno con una responsabilità unica:

- `report.ts` — modella il report statistico e la sua logica di aggiornamento;
- `scanner.ts` — implementa la logica di traversal asincrono del file system;
- `path.ts` — fornisce un tipo validato per i percorsi del file system;
- `sizes.ts` — utility per la conversione delle unità di dimensione dei file;
- `benchmark.ts` — utility per la misurazione dei tempi di esecuzione;
- `index.ts` — façade pubblica della libreria verso i consumer esterni.

Pattern di incapsulamento: factory function e interfaccia pubblica. Sia `FsStatReport` che `FsStatScanner` adottano un pattern idiomatico TypeScript per l'information hiding: la classe concreta (`FsStatReportImpl`, `FsStatScannerImpl`) è dichiarata `class` privata al modulo e non viene mai esportata. L'unico punto di accesso è la *factory function* omonima, che restituisce l'interfaccia pubblica:

```
1 export function FsStatScanner(): FsStatScanner {
2   return new FsStatScannerImpl();
3 }
```

Il consumer non conosce mai la classe concreta né può accedere ai suoi membri privati: interagisce esclusivamente tramite l'interfaccia, che espone solo i metodi necessari (`getReport`, `updateReport`, `dto`). Questo garantisce un disaccoppiamento totale tra interfaccia e implementazione.

Il Branded Type Path. Il modulo `path.ts` definisce un *Branded Type* per i percorsi del file system:

```
1 export type Path = string & { readonly __brand: unique symbol };
```

Si tratta di un *phantom type*: a runtime `Path` è semplicemente una stringa, ma a compile-time il type system di TypeScript impone che solo path prodotti da `createPath()` o `concatPaths()` — che normalizzano e validano l'input — possano essere passati alle funzioni interne dello scanner. Qualsiasi stringa grezza produce un errore di tipo a compile-time, eliminando un'intera categoria di bug senza alcun overhead a runtime.

Separazione del modello interno dall'API esterna: il metodo `dto()`. `FsStatReportImpl` mantiene internamente uno stato mutabile (`_numOfFiles`, `_sizeBands`), aggiornato ad ogni chiamata di `updateReport()`. Tuttavia, il consumer esterno non riceve mai accesso diretto a questo stato: il metodo `dto()` produce uno snapshot immutabile nel formato `FsStatReportDTO`, definito come tipo separato nel package dei tipi condivisi:

```
1 dto(): FsStatReportDTO {
2   return {
3     numOfFiles: this.numOfFiles,
4     sizeBands: [...this.sizeBands], // copia difensiva
5   };
6 }
```

La copia difensiva dell'array (`[...this.sizeBands]`) garantisce che il DTO sia uno snapshot indipendente: eventuali aggiornamenti successivi al report non alterano il risultato già consegnato al consumer.

Il flusso di traversal: `FsStatScannerImpl`. Il metodo pubblico `getReport()` istanzia un `FsStatReport` vuoto e avvia la traversal ricorsiva tramite `traverseDirectory()`, restituendo la `Promise` che si risolve al termine dell'intera scansione.

Il metodo `traverseDirectory()` costituisce il nucleo del design asincrono. Il suo flusso segue tre fasi:

1. **Guard sulla blacklist:** prima di qualsiasi operazione I/O, il path viene verificato contro un `Set` di percorsi di sistema esclusi (`/proc`, `/sys`, `/dev`, `/run` e altri). Se il path è nella blacklist, la funzione ritorna immediatamente senza effettuare alcuna lettura, prevenendo loop infiniti o comportamenti non deterministici su filesystem virtuali.
2. **Lettura della directory:** `fs.readdir()` con l'opzione `withFileTypes: true` restituisce direttamente le informazioni sul tipo di ciascuna voce (file o directory), evitando una chiamata aggiuntiva a `fs.stat()` per discriminarle. In caso di errore di accesso

(EACCES/EPERM), il `.catch()` restituisce un array vuoto, permettendo alla scansione di proseguire sui nodi accessibili.

3. **Fan-out parallelo con `Promise.all`:** file e sottodirectory vengono processati simultaneamente in un'unica barriera di sincronizzazione:

```
1 await Promise.all([
2   ...files.map((file) => exploreFile(file.name)),
3   ...directories.map((dir) => exploreDir(dir.name)),
4 ]);
```

Tutte le chiamate a `fs.stat()` sui file e tutte le esplorazioni ricorsive delle sottodirectory vengono avviate in parallelo. L'event loop gestisce il completamento di ciascuna operazione in modo indipendente: non appena `fs.stat()` completa per un file, la relativa callback aggiorna il report senza attendere gli altri. `Promise.all` funge da barriera: `traverseDirectory` si risolve solo quando *tutte* le operazioni figlie sono terminate.

Gestione degli errori a due livelli. L'implementazione distingue esplicitamente due categorie di errori tramite il metodo `ignoreAccessError()`:

- **Errori di permesso (EACCES, EPERM):** vengono silenziosamente ignorati. La scansione prosegue sui path accessibili, poiché è normale che alcune directory di sistema siano protette.
- **Errori strutturali (ENOENT, ENOTDIR):** vengono convertiti in eccezioni tipizzate tramite `parseScanningError()` e propagati al consumer come rigetto della `Promise` restituita da `getReport()`, segnalando una condizione anomala che richiede attenzione.

La façade pubblica: `index.ts`. Il modulo `index.ts` funge da unico punto di accesso alla libreria per i consumer esterni. Crea internamente un'unica istanza di `FsStatScanner` ed espone `getFSReport` come funzione standalone tramite `bind`, nascondendo completamente l'esistenza della classe:

```
1 const scanner = FsStatScanner();
2 const getFSReport = scanner.getReport.bind(scanner);
```

Il consumer può importare `getFSReport` direttamente come funzione, senza mai istanziare né conoscere `FsStatScanner`. Questa scelta rispecchia lo stile funzionale prevalente nell'ecosistema Node.js, dove si preferisce esporre funzioni piuttosto che oggetti con stato esplicito.

CLI e misurazione delle prestazioni. Il modulo `main.ts` implementa la CLI: legge il path da `process.argv[2]` con fallback alla directory corrente (`process.cwd()`), configura i parametri (`maxSize = 100 MB`, `nb = 5`) tramite l'utility `FileSize.megabyte()`, e misura il tempo di esecuzione con `benchmark()`. Quest'ultima funzione, basata su `process.hrtime.bigint()` per precisione al nanosecondo, restituisce sia il risultato che la durata in millisecondi, che vengono poi stampati in formato JSON indentato tramite `JSON.stringify(report, null, 2)`.

3.3 Design dell'implementazione RX

Il design reattivo stravolge l'architettura passando dalla "richiesta di dati" (pull) alla "reazione agli eventi" (push). La libreria espone un `Flowable<FSReport>` o `Observable<FSReport>`.

- **Aggiornamenti Dinamici:** La GUI si limita a effettuare una `subscribe()` sullo stream, ricevendo un nuovo snapshot del report ogniqualvolta la pipeline reattiva emette un aggiornamento.

- **Cancellazione:** Per fermare il task è sufficiente invocare il metodo `dispose()` (o disiscriversi dallo stream); il framework RX propaga istantaneamente e in modo automatico il segnale di interruzione alla sorgente (l'esploratore delle directory), chiudendo pulitamente le risorse aperte.

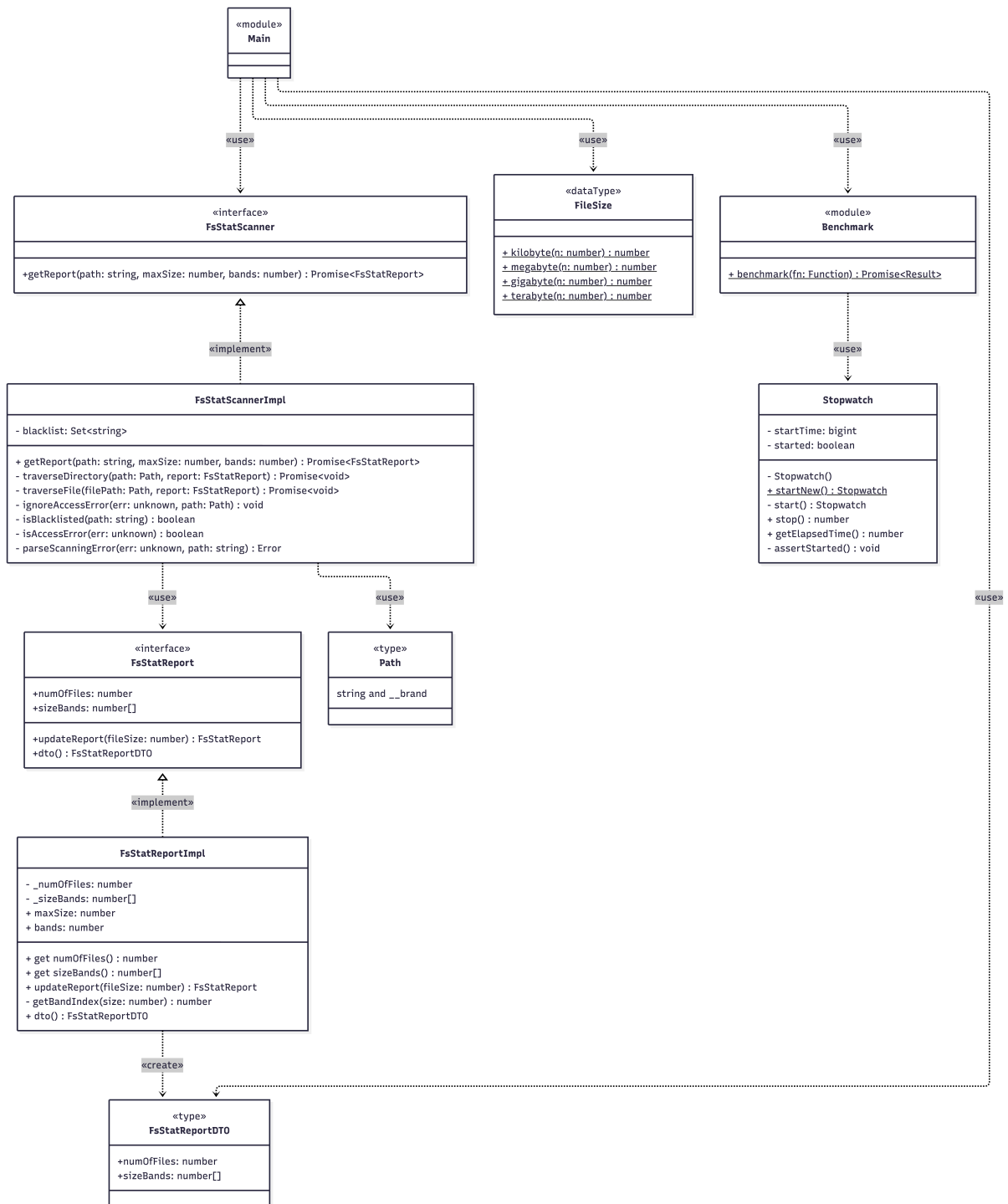


Figura 2: Diagramma UML delle classi dell'implementazione con Event Loop (TypeScript).